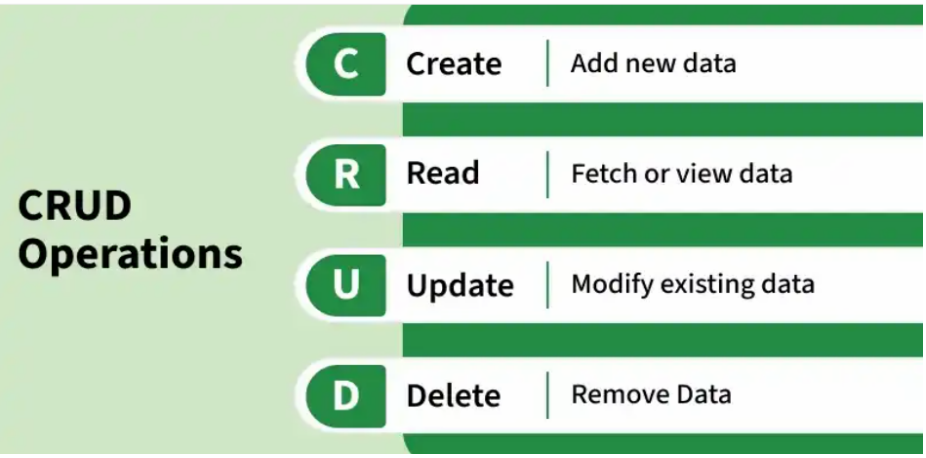
# MongoDB CRUD Operations



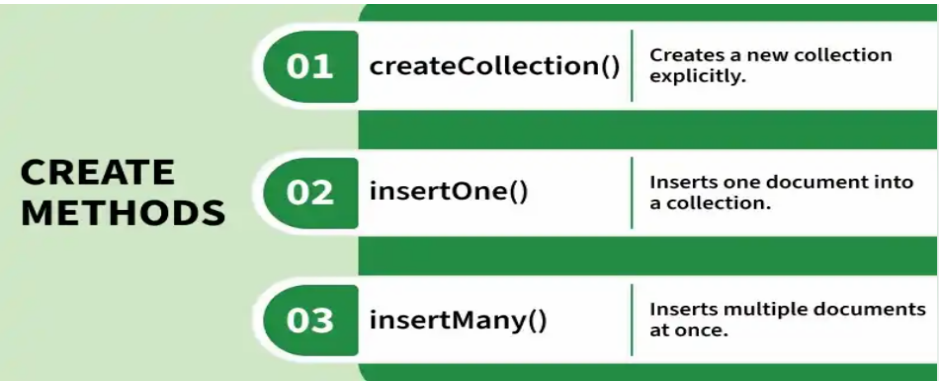

## Create, View, Update, and Delete Documents

You can use the Atlas UI to manage documents inside your collections. Documents are individual records in a MongoDB collection and are the basic unit of data in MongoDB.

Viewing documents and collections in the Atlas UI can provide a high-level overview of your database schema. You can use the Atlas UI to ensure you are following MongoDB's core data modeling concepts, such as utilizing embedded documents and arrays.

CRUD operations are fundamental database actions in MongoDB that enable users to insert, read, update, and delete documents within collections.

**Create Operations**

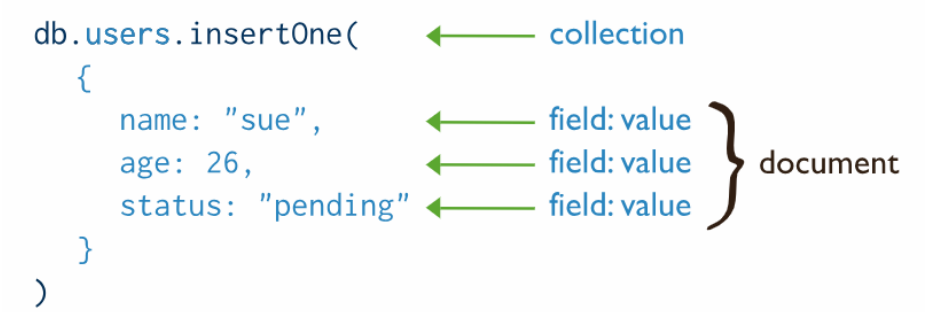



Create (insert) operations are used to add new documents to a collection, creating the collection automatically if it does not already exist.

- Adds new documents to a collection.
- Automatically creates the collection if missing.
- Generates a unique _id if not provided.
- Supports inserting single or multiple documents.

MongoDB provides the following methods to insert documents into a collection:

- db.collection.insertOne()
- db.collection.insertMany()




Example 1: We are creating a student collection in the database using the db.createCollection() method.

```
test> use GeeksforGeeks
switched to db GeeksforGeeks
GeeksforGeeks> db.createCollection("faculty")
{ ok: 1 }
```

Example 2: We are inserting details of a single student in the form of document in the student collection using db.collection.insertOne() method.

```
test> use GeeksforGeeks
switched to db GeeksforGeeks
GeeksforGeeks> db.student.insertOne({
... name : "Alen",
... age : 20,
... branch : "CSE",
... course : "C++ STL",
... mode : "online",
... paid : true,
... amount : 1499
GeeksforGeeks>
{
  acknowledged: true,
  insertedId: ObjectId('6981cc70cfa7f77c71628ca0')
GeeksforGeeks>
```

Example 3: We are inserting details of the multiple students in the form of documents in the student collection using db.collection.insertMany() method.

```
GeeksforGeeks> db.student.insertMany([
... {
... name : "Kim",
... age : 20,
... branch : "CSE",
... mode : "online",
... paid : true,
... amount : 1499
... },
...
... name : "Ron",
... age : 21,
... branch : "CSE",
... course : "C++ STL",
... mode : "online",
... paid : true,
... amount : 1499
... }
...
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6981ce59cfa7f77c71628ca1'),
    '1': ObjectId('6981ce59cfa7f77c71628ca2')
  }
}
```

**FOR YOUR REFERENCE PURPOSE ONLY:**

To add one or more documents to a collection through the Atlas UI, you can specify the documents to insert from scratch or you can clone an existing document and modify its fields and values as needed.

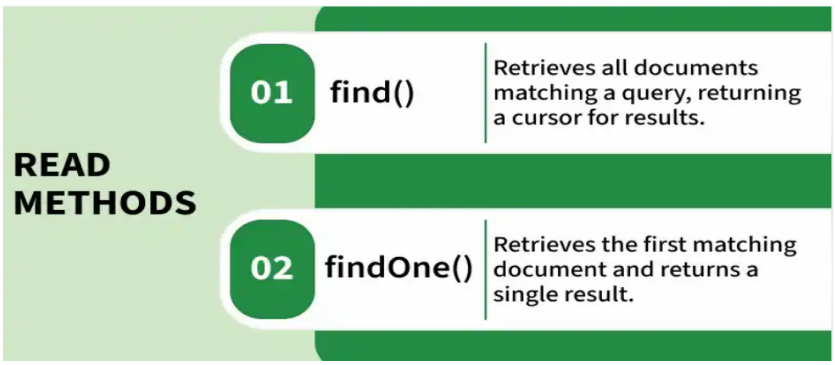
To insert documents into your collection, use one of the following methods in Atlas:

- **Shell Syntax Mode**: Write or paste documents in the MongoDB shell syntax, which allows unquoted field names and type constructors such as ObjectId() and ISODate(). Use this mode to insert multiple documents at once as an array.
- **Field-by-Field Editor**: Select individual field values and types for each inserted document. You can only insert one document at a time using this method.
- **JSON Mode**: Write or paste documents in Extended JSON format, which requires quoted field names and represents BSON types as wrappers such as {"$oid": "..."}. Use this mode to insert multiple documents at once as an array.

**Read Operations**

Read operations are used to retrieve or query documents from a MongoDB collection.

- Fetch documents from a collection.
- Support filtering, projection, sorting, and limiting results.
- Used to read and analyze stored data.




**Examples 1:** We are retrieving the details of students from the student collection using db.collection.find() method.

```
GeeksforGeeks> db.student.find().pretty()
[
  {
    _id: ObjectId('6981cc70cfa7f77c71628ca0'),
    name: 'Alen',
    age: 20,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca1'),
    name: 'Kim',
    age: 20,
    branch: 'CSE',
    mode: 'online',
    paid: true,
    amount: 1499
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca2'),
    name: 'Ron',
    age: 21,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499
  }
]
```

***Note:*** *pretty() method is used to decorate the result such that it is easy to read.*

**Example 2:** We are retrieving the details of a single student from the student collection using the db.collection.findOne() method.

```
GeeksforGeeks> db.student.findOne({name: "Alen"})
{
  _id: ObjectId('6981cc70cfa7f77c71628ca0'),
  name: 'Alen',
  age: 24,
  branch: 'CSE',
  course: 'C++ STL',
  mode: 'online',
  paid: true,
  amount: 1499
}
```

You can specify query filters or criteria that identify the documents to return.




In **MongoDB shell (mongosh)**, you can find records in multiple ways using the find() method with different parameters, filters, projections, and options:

**Basic Find**

Javascript

```
// Find all documents in a collection
db.users.find();
```

**Find with Filter**

Javascript

```
// Find documents where age is greater than 25
db.users.find({ age: { $gt: 25 } });
```

```
// Find documents where name is "John"
db.users.find({ name: "John" });
```

**Find with Projection (Select Specific Fields)**

Javascript

```
// Show only name and age fields, hide _id
db.users.find(
  { age: { $gt: 25 } },
  { name: 1, age: 1, _id: 0 }
);
```

**Find with Multiple Conditions**

Javascript

```
// AND condition
db.users.find({ age: { $gt: 25 }, city: "New York" });

// OR condition
db.users.find({
  $or: [
    { age: { $lt: 20 } },
    { city: "London" }
  ]
});
```

**Find with Sorting**

Javascript

```
// Sort by age ascending
db.users.find().sort({ age: 1 });

// Sort by age descending
db.users.find().sort({ age: -1 });
```

**Find with Limit & Skip**

Javascript

```
// Limit results to 5 documents
db.users.find().limit(5);
```

```
// Skip first 10 documents
db.users.find().skip(10);
```

**Find with Aggregation-like Query**

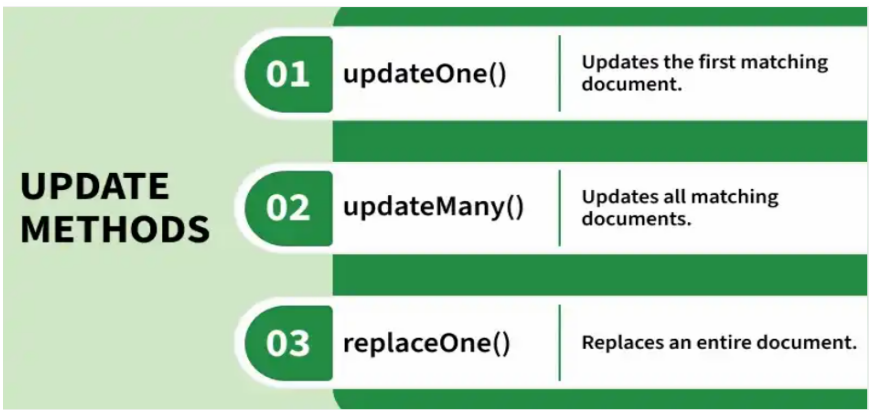
Javascript

```
// Find and sort with limit
db.users.find({ age: { $gt: 25 } })
    .sort({ age: -1 })
    .limit(3);
```

**Update Operations**

Update operations are used to modify existing documents in a MongoDB collection based on specified query conditions.

- Update single or multiple matching documents.
- Can replace entire documents if required.




**Example 1:** We are updating the age of Alen in the student collection using db.collection.updateOne() method.

```
GeeksforGeeks> db.student.updateOne({name: "Alen"},{$set:{age: 24}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
GeeksforGeeks> db.student.find().pretty()
[
  {
    _id: ObjectId('6981cc70cfa7f77c71628ca0'),
    name: 'Alen',
    age: 24,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca1'),
    name: 'Kim',
    age: 20,
    branch: 'CSE',
    mode: 'online',
    paid: true,
    amount: 1499
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca2'),
    name: 'Ron',
    age: 21,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499
  }
]
```

**Example 2:** We are updating the year of course in all the documents in the student collection using db.collection.updateMany() method.

```
GeeksforGeeks> db.student.updateMany({}, {$set: {year: 2020}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
GeeksforGeeks> db.student.find().pretty()
[
  {
    _id: ObjectId('6981cc70cfa7f77c71628ca0'),
    name: 'Alen',
    age: 24,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499,
    year: 2020
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca1'),
    name: 'Kim',
    age: 20,
    branch: 'CSE',
    mode: 'online',
    paid: true,
    amount: 1499,
    year: 2020
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca2'),
    name: 'Ron',
    age: 21,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499,
    year: 2020
  }
]
```

**Example 3:** We are replacing the complete details of a single student document in the student collection using the db.student.replaceOne() method.

```
GeeksforGeeks> db.student.replaceOne({name: "Kim"},{name: "Kim", age: 25, branch: "ECE", mode: "offline", paid: false, amount: 0})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
GeeksforGeeks> db.student.find({name: "Kim"})
[
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca1'),
    name: 'Kim',
    age: 25,
    branch: 'ECE',
    mode: 'offline',
    paid: false,
    amount: 0
  }
]
```

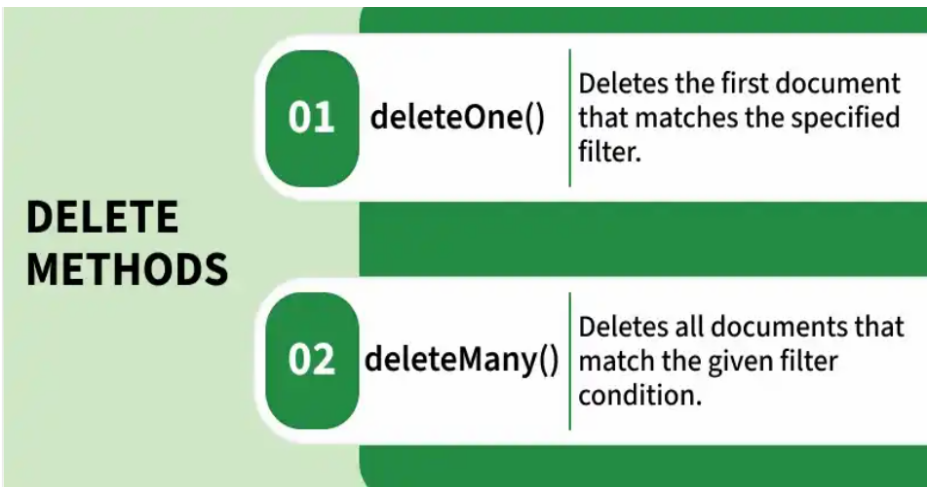
**Difference between updateOne(), updateMany() and replaceOne()**

| Feature | updateOne() | updateMany() | replaceOne() |
|---|---|---|---|
| Purpose | Updates specific fields | Updates specific fields | Replaces the entire document |
| Documents affected | Only one | All matching documents | Only one |
| Uses $set | Yes | Yes | No |

**Delete Operations**

Delete operations are used to remove documents from a MongoDB collection.

- Delete documents based on specific conditions.
- Can remove a single document or multiple documents
- Supports deleting all documents in a collection



**Example 1:** We are deleting a document from the student collection using db.collection.deleteOne() method.

```
GeeksforGeeks> db.student.deleteOne({name: "Alen"})
{ acknowledged: true, deletedCount: 1 }
GeeksforGeeks> db.student.find().pretty()
[
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca1'),
    name: 'Kim',
    age: 25,
    branch: 'ECE',
    mode: 'offline',
    paid: false,
    amount: 0
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca2'),
    name: 'Ron',
    age: 21,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499,
    year: 2020
  }
]
```

**Example 2:** We are deleting multiple documents from the student collection using db.collection.deleteMany() method (all documents are deleted only if an empty filter {} is used).

```
GeeksforGeeks> db.student.find().pretty()
[
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca1'),
    name: 'Kim',
    age: 25,
    branch: 'ECE',
    mode: 'offline',
    paid: false,
    amount: 0
  },
  {
    _id: ObjectId('6981ce59cfa7f77c71628ca2'),
    name: 'Ron',
    age: 21,
    branch: 'CSE',
    course: 'C++ STL',
    mode: 'online',
    paid: true,
    amount: 1499,
    year: 2020
  }
]
GeeksforGeeks> db.student.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
```